

National Institute of Electronics and Information Technology, Chennai

Online Internship in
RTL Design Using Verilog HDL

Project Report on

Implementation of 5-Stage Pipelined RISC Processor Using Verilog HDL

Submitted By

Aditya Raj

at

July 2026 – August 2026

About NIELIT

National Institute of Electronics and Information Technology (NIELIT), headquartered in New Delhi is the capacity building arm of the Ministry of Electronics and Information Technology (MeitY), mandated to carry out HR development and related activities in Information, Electronics and Communication Technology (IECT).

Since its inception in 2010, NIELIT Chennai has established itself as a premier institution providing affordable quality education as per the job market requirements for candidates primarily from Tamil Nadu. As of date NIELIT Chennai has imparted skill training to more than 30, 000 youth of this region and conducted digital literacy certification for more than 25,000 candidates. NIELIT Chennai handles the activities of the NIELIT in Tamil Nadu, Andhra, Telangana, and the Andaman and Nicobar Islands.

Presently NIELIT Chennai is implementing the following projects for Tamil Nadu: ISEA (Information Security Education and Awareness), Future Skills PRIME capacity building projects (3D Printing and Additive Manufacturing, Robotic Process Automation, IoT - technology streams), and the aspirational district projects for SC/ST candidates at Ramanathapuram and Virudhunagar funded by MeitY, Government of India. The central also received funding through the electronics product design and product testing capacity building project funded by MeitY.

Other than training, the centre also provides services like; Private Cloud Setup using Citrix, vSphere, Red Hat & FOSS Cloud, Hyper-converged Infrastructure (HCI) and Virtual Desktop Infrastructure (VDI) Server Setup, High-Performance Computing (HPC) Setup with 100G/200G connectivity, Server and Virtual Lab Setup for BigData, AI & Information Security, and Digital & Mobile Forensics Service.

NIELIT has also set up a Virtual Academy, a common End-to-End Platform to conduct Online training courses including a virtual lab environment in the areas of IECT from foundation to expert level targeting from school students to working professionals.

INDEX

Contents	Page No.
ACKNOWLEDGEMENT	1
ABSTRACT	2
LIST OF FIGURES	3
CHAPTER 1: INTRODUCTION	4-5
CHAPTER 2: LITERATURE	6
CHAPTER 3: HARDWARE TOOLS AND COMPONENT	7
CHAPTER 4: SOFTWARE TOOLS AND LANGUAGES	8-10
CHAPTER 5: IMPLEMENTATION	11-12
CHAPTER 5.1: FLOW CHART	13
CHAPTER 6: RESULTS AND DISCUSSION	14-21
CHAPTER 7: CONCLUSION AND DISCUSSION	22
CHAPTER 8: REFERENCE	23

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have supported and guided me throughout the completion of this project on as part of the Online Internship in RTL Design Using Verilog HDL conducted by the National Institute of Technology and Applied Research (NIELIT), Chennai.

First and foremost, I would like to thank my course instructor, Ishant Kumar Bajpai (Scientist 'D') , for their invaluable guidance, constructive feedback, and constant encouragement throughout the project. Their expertise in the field of Chip Design has been a key factor in the successful completion of this report.

I would also like to extend my appreciation to the NIELIT Chennai for offering this comprehensive online course, providing us with high-quality learning resources and support despite the challenges of remote learning.

A special thanks to my fellow participants in the online course for the engaging discussions and knowledge-sharing that enriched my learning experience. The collaborative nature of this course has significantly enhanced the depth and quality of my work.

Finally, I would like to thank all the authors, researchers, and resources referenced in this report for their contributions to the field of Chip Design, which formed the foundation of my project.

Thank you to everyone who has been part of this learning journey.

ABSTRACT

The main objective of this project is to design and implement a **5-Stage Pipelined RISC Processor** using **Verilog HDL**. The project was carried out as part of the **Online Internship in RTL Design Using Verilog HDL** organized by **NIELIT Chennai**.

A RISC (Reduced Instruction Set Computer) processor uses a simple instruction set, which helps in faster execution and better performance. In this project, the processor is divided into five stages: **Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB)**. These stages allow multiple instructions to be processed simultaneously, improving the overall efficiency of the processor.

The design is written in Verilog HDL and verified using **Xilinx Vivado**. Behavioural simulation, synthesis, static timing analysis, implementation, and post-implementation simulations are performed to verify the functionality and timing performance of the design.

This project helped me understand RTL design flow, Verilog coding, digital logic design, and processor architecture. It also provided practical experience in using Vivado for designing and verifying digital hardware.

LIST OF FIGURES

S.NO.	TITLE	PAGE NO.
1	Basic Block Diagram of a 5-Stage Pipelined RISC Processor	4
2	Basic Comparison between RISC and CISC Architecture	6
3	Digilent Basys 3 FPGA Development Board	7
4	Xilinx Vivado Design Suite	8
5	RTL Schematic Generated in Vivado	12
6	Flow Chart of the Proposed RISC Processor	13
7	Behavioural Simulation Waveform of the 5-Stage Pipelined RISC Processor	14
8	Synthesized RTL Schematic of the 5-Stage Pipelined RISC Processor	15
9	Post-Synthesis Functional Simulation Waveform	16
10	Post-Synthesis Timing Simulation Waveform	17
11	Static Timing Analysis (Timing Summary Report)	18
12	FPGA Device Layout after Implementation	19
13	Post-Implementation Functional Simulation Waveform	20
14	Post-Implementation Timing Simulation Waveform	21

CHAPTER 1

INTRODUCTION

A **Reduced Instruction Set Computer (RISC)** is a processor architecture that uses a small and simple set of instructions. The main purpose of a RISC processor is to execute instructions quickly with better performance and lower hardware complexity. Due to its simple instruction format, RISC processors are widely used in embedded systems, microcontrollers, and modern digital devices.

In this project, a **5-Stage Pipelined RISC Processor** is designed using **Verilog HDL**. Pipelining is a technique in which multiple instructions are processed simultaneously by dividing the execution into different stages. This improves the overall speed and increases the instruction throughput without increasing the clock frequency.

The processor consists of five stages: **Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB)**. Each stage performs a specific task, and pipeline registers are used to transfer data from one stage to the next. This approach allows one instruction to enter the pipeline every clock cycle.

Verilog HDL is used to implement the complete RTL design of the processor. The design is simulated and verified using **Xilinx Vivado**. After successful simulation, synthesis and implementation are performed to verify the functionality and timing performance of the processor.

The main objective of this project is to understand the RTL design flow, processor architecture, and pipelining concept through practical implementation using Verilog HDL.

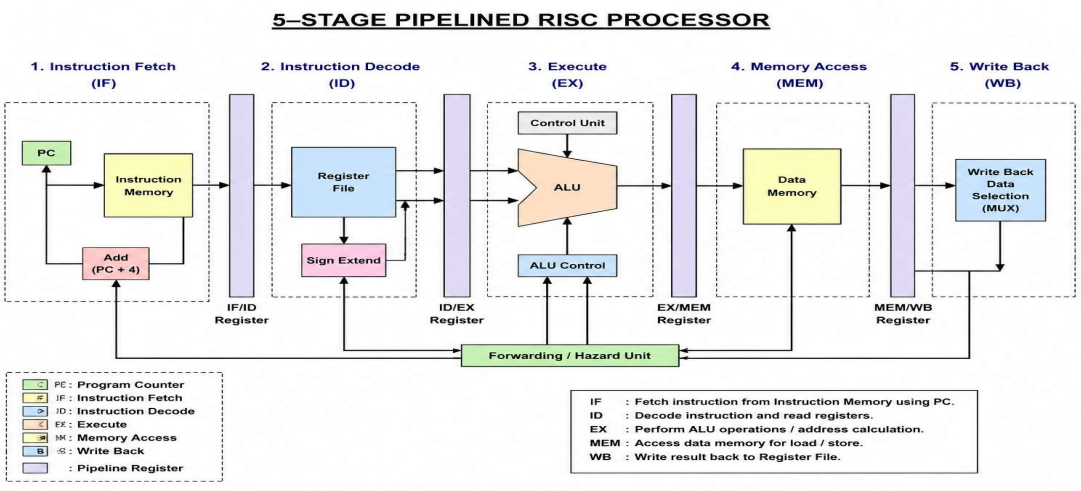


Figure 1: Basic Block Diagram of a 5-Stage Pipelined RISC Processor

CHAPTER 1

Objectives of the Project

The main objectives of this project are:

To design a **5-Stage Pipelined RISC Processor** using Verilog HDL.

To understand the working of different pipeline stages such as Instruction Fetch, Instruction Decode, Execute, Memory Access, and Write Back.

To implement the processor using RTL design methodology.

To verify the design using Behavioural Simulation in Xilinx Vivado.

To perform Synthesis, Static Timing Analysis (STA), and Implementation of the design.

To analyse the performance of a pipelined processor and understand its advantages over a non-pipelined processor.

Advantages of Pipelining

A pipelined processor provides better performance by executing multiple instructions at the same time. It increases instruction throughput and makes better use of hardware resources. Pipelining also reduces the average execution time of instructions and improves the overall efficiency of the processor.

Applications

The RISC processor architecture is widely used in embedded systems, microcontrollers, consumer electronics, networking devices, automotive systems, and modern processors. The concepts of RTL design and pipelining are also used in FPGA and ASIC development.

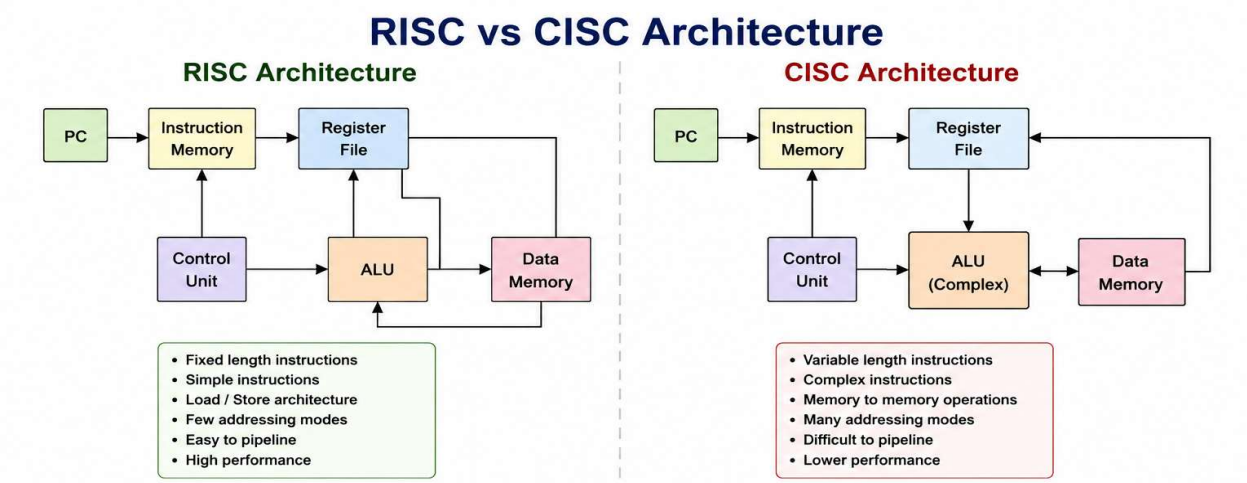
CHAPTER 2
LITERATURE

The design of RISC processors has been an important area of research in digital system design. Compared to Complex Instruction Set Computer (CISC) processors, RISC processors use fewer and simpler instructions. This makes the processor easier to design and allows instructions to execute more efficiently.

Many researchers have implemented RISC processors using Verilog HDL because it provides an effective way to describe and verify digital hardware. Verilog allows designers to develop hardware at the Register Transfer Level (RTL), making the design easier to simulate, test, and synthesize before hardware implementation.

Pipeline architecture is one of the most common techniques used to improve processor performance. A 5-stage pipelined processor divides instruction execution into five stages: Instruction Fetch, Instruction Decode, Execute, Memory Access, and Write Back. This approach increases instruction throughput by processing multiple instructions at the same time.

In this project, the same concept is implemented using Verilog HDL. The processor is verified through behavioural simulation, synthesis, static timing analysis, and implementation using Xilinx Vivado. The project provides practical knowledge of RTL design and helps in understanding the complete FPGA design flow.



Feature	RISC	CISC
Instruction Set	Small and simple	Large and complex
Instruction Length	Fixed	Variable
Addressing Modes	Few	Many
Execution Time	Most instructions in 1 clock cycle	Multiple clock cycles
Memory Access	Load / Store only	Memory to Memory also allowed
Hardware Complexity	Simple	Complex
Performance	High	Lower
Pipelining	Easy	Difficult
Examples	ARM, MIPS, RISC-V, SPARC	x86, 68000, VAX

Figure 2: Basic Comparison between RISC and CISC Architecture

CHAPTER 3
HARDWARE TOOLS AND COMPONENT

Hardware Tools and Components

The hardware and software resources used in this project are listed below. These tools were used to design, simulate, and verify the 5-Stage Pipelined RISC Processor.

1. Laptop

A laptop was used for designing and simulating the Verilog HDL code. The system should have at least an Intel Core i3/i5 processor, 8 GB RAM, and 20 GB of free storage for smooth operation of Xilinx Vivado.

2. FPGA Board (Optional)

An FPGA board can be used to implement and test the processor design on real hardware. In this project, the complete design was verified using simulation in Xilinx Vivado. Therefore, an FPGA board was not mandatory.

3. Display Device

A laptop display was used to view the RTL schematic, simulation waveforms, synthesis reports, and timing analysis.

The above hardware components were sufficient to complete the RTL design, behavioural simulation, synthesis, and implementation of the processor.

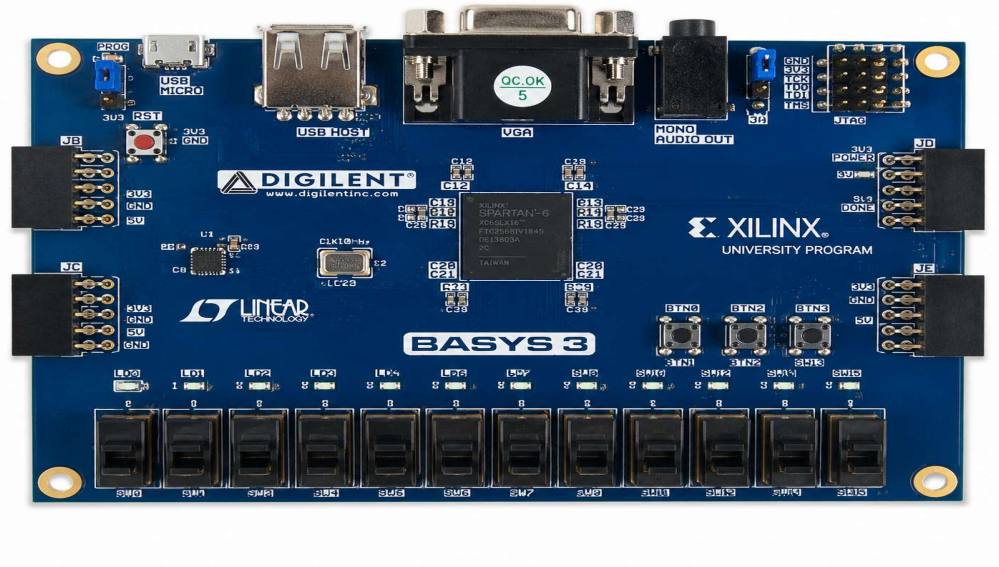


Figure 3: Digilent Basys 3 FPGA Development Board

CHAPTER 4
SOFTWARE TOOLS AND LANGUAGES

The software tools and programming language used in this project are described below.

1. Verilog HDL

Verilog HDL is a Hardware Description Language used to design and describe digital circuits at the Register Transfer Level (RTL). It is widely used for FPGA and ASIC design because it supports simulation, synthesis, and hardware implementation.

2. Xilinx Vivado Design Suite

Xilinx Vivado is the main software used in this project. It provides an integrated environment for writing Verilog code, performing behavioural simulation, synthesis, implementation, and static timing analysis. It also generates RTL schematics and timing reports for hardware verification.

3. Windows Operating System

The project was developed on the Windows operating system. It provides a stable platform for running Vivado and managing the project files efficiently.

4. Simulation and Verification

Behavioural simulation was performed to verify the functionality of the processor before synthesis. After synthesis and implementation, timing analysis was carried out to ensure that the design met the required timing constraints.

The combination of Verilog HDL and Xilinx Vivado provides a complete RTL design flow from coding to hardware implementation.

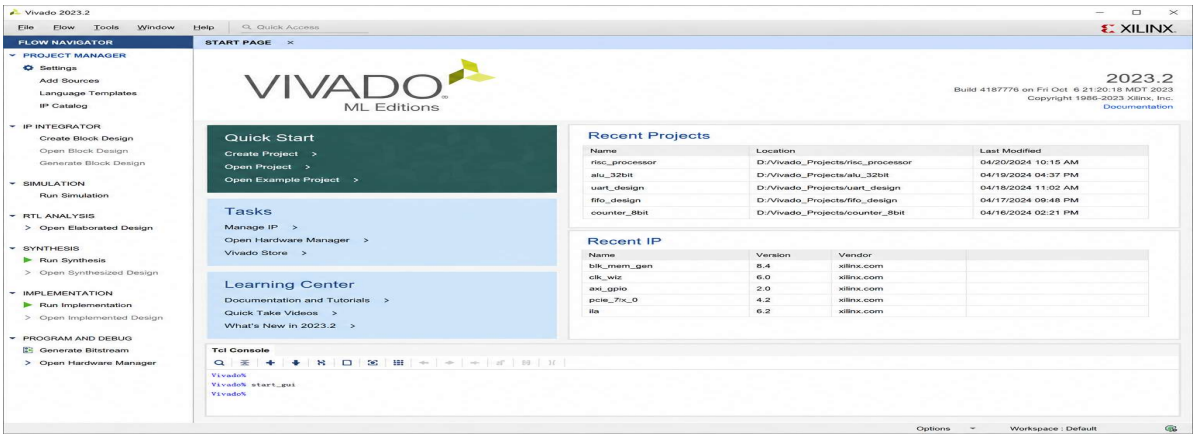


Figure 4.1: Xilinx Vivado Design Suite

CHAPTER 4.1
VERILOG CODE DESIGN

top_risc_processor.v | top_risc_processor_tb.v | instruction_memory.v | regist

C:/Users/aditya/RISC/RISC.srcs/sources_1/new/top_risc_processor.v

```
22
23      module top_risc_processor(
24          input clk,
25          input reset,
26          output [31:0] pc_out,
27          output [31:0] instruction_out,
28          output [31:0] alu_result_out
29      );
30      wire [31:0] pc;
31      wire [31:0] next_pc;
32      wire [31:0] instruction;
33
34      wire [31:0] id_pc;
35      wire [31:0] id_instruction;
36
37      wire [5:0] opcode;
38      wire [4:0] rs;
39      wire [4:0] rt;
40      wire [4:0] rd;
41      wire [15:0] immediate;
42
43      wire [31:0] read_data1;
44      wire [31:0] read_data2;
45      wire [31:0] sign_ext_data;
46
47      wire [31:0] ex_pc;
48      wire [31:0] ex_read_data1;
49      wire [31:0] ex_read_data2;
50      wire [31:0] ex_immediate;
51
52      wire [4:0] ex_rd;
53      wire [4:0] mem_rd;
54      wire [4:0] wb_rd;
55
56      wire [31:0] alu_result;
57      wire zero;
58
59      wire [31:0] mem_alu_result;
60      wire [31:0] mem_write_data;
61      wire [31:0] mem_read_data;
62
63      wire [31:0] wb_alu_result;
64      wire [31:0] wb_read_data;
65
```

top_risc_processor.v | top_risc_processor_tb.v | instruction_memor

C:/Users/aditya/RISC/RISC.srcs/sources_1/new/top_risc_processor.v

```
64      wire [31:0] wb_read_data;
65
66      wire RegWrite;
67      wire MemRead;
68      wire MemWrite;
69      wire MemtoReg;
70      wire ALUSrc;
71      wire Branch;
72      wire [2:0] ALU_Control;
73
74      wire RegWrite_EX;
75      wire MemRead_EX;
76      wire MemWrite_EX;
77      wire MemtoReg_EX;
78      wire ALUSrc_EX;
79      wire Branch_EX;
80      wire [2:0] ALU_Control_EX;
81
82      wire RegWrite_MEM;
83      wire MemRead_MEM;
84      wire MemWrite_MEM;
85      wire MemtoReg_MEM;
86
87      wire RegWrite_WB;
88      wire MemtoReg_WB;
89      assign next_pc = pc + 4;
90
91      assign opcode = id_instruction[31:26];
92      assign rs = id_instruction[25:21];
93      assign rt = id_instruction[20:16];
94      assign rd = id_instruction[15:11];
95      assign immediate = id_instruction[15:0];
96      program_counter PC(
97          .clk(clk),
98          .reset(reset),
99          .next_pc(next_pc),
100         .pc(pc)
101     );
102
103     instruction_memory IM(
104         .address(pc),
105         .instruction(instruction)
106     );
107
```

CHAPTER 4.2
VERILOG CODE DESIGN

top_risc_processor.v | top_risc_processor_tb.v | instruction_memory

C:/Users/aditya/RISC/RISC.srcs/sources_1/new/top_risc_processor.v

```
109             .clk(clk),
110             .reset(reset),
111             .IF_PC(pc),
112             .IF_Instruction(instruction),
113             .ID_PC(id_pc),
114             .ID_Instruction(id_instruction)
115         );
116
117         control_unit CU(
118             .opcode(opcode),
119             .RegWrite(RegWrite),
120             .MemRead(MemRead),
121             .MemWrite(MemWrite),
122             .MemtoReg(MemtoReg),
123             .ALUSrc(ALUSrc),
124             .Branch(Branch),
125             .ALU_Control(ALU_Control)
126         );
127
128         register_file RF(
129             .clk(clk),
130             .reg_write(RegWrite_WB),
131             .rs(rs),
132             .rt(rt),
133             .rd(wb_rd),
134             .write_data(wb_alu_result),
135             .read_data1(read_data1),
136             .read_data2(read_data2)
137         );
138
139         sign_extend SE(
140             .immediate(immediate),
141             .extended_immediate(sign_ext_data)
142         );
143         ID_EX IDEX(
144             .clk(clk),
145             .reset(reset),
146
147             .ID_PC(id_pc),
148             .ID_ReadData1(read_data1),
149             .ID_ReadData2(read_data2),
150             .ID_Immediate(sign_ext_data),
151             .ID_rd(rd),
152
```

top_risc_processor.v | top_risc_processor_tb.v | instruction_mem

C:/Users/aditya/RISC/RISC.srcs/sources_1/new/top_risc_processor.v

```
152
153             .RegWrite_in(RegWrite),
154             .MemRead_in(MemRead),
155             .MemWrite_in(MemWrite),
156             .MemtoReg_in(MemtoReg),
157             .ALUSrc_in(ALUSrc),
158             .Branch_in(Branch),
159             .ALU_Control_in(ALU_Control),
160
161             .EX_PC(ex_pc),
162             .EX_ReadData1(ex_read_data1),
163             .EX_ReadData2(ex_read_data2),
164             .EX_Immediate(ex_immediate),
165             .EX_rd(ex_rd),
166
167             .RegWrite_out(RegWrite_EX),
168             .MemRead_out(MemRead_EX),
169             .MemWrite_out(MemWrite_EX),
170             .MemtoReg_out(MemtoReg_EX),
171             .ALUSrc_out(ALUSrc_EX),
172             .Branch_out(Branch_EX),
173             .ALU_Control_out(ALU_Control_EX)
174         );
175
176         alu ALU(
177             .A(ex_read_data1),
178             .B(ex_read_data2),
179             .ALU_Control(ALU_Control_EX),
180             .Result(alu_result),
181             .Zero(zero)
182         );
183         EX_MEM EXMEM(
184             .clk(clk),
185             .reset(reset),
186
187             .EX_ALU_Result(alu_result),
188             .EX_WriteData(ex_read_data2),
189             .EX_rd(ex_rd),
190
191             .RegWrite_in(RegWrite_EX),
192             .MemRead_in(MemRead_EX),
193             .MemWrite_in(MemWrite_EX),
194             .MemtoReg_in(MemtoReg_EX),
195
```

top_risc_processor.v | top_risc_processor_tb.v | instruction_memory.v

C:/Users/aditya/RISC/RISC.srcs/sources_1/new/top_risc_processor.v

```
194             .MemtoReg_in(MemtoReg_EX),
195
196             .MEM_ALU_Result(mem_alu_result),
197             .MEM_WriteData(mem_write_data),
198             .MEM_rd(mem_rd),
199
2000            .RegWrite_out(RegWrite_MEM),
2001            .MemRead_out(MemRead_MEM),
2002            .MemWrite_out(MemWrite_MEM),
2003            .MemtoReg_out(MemtoReg_MEM)
2004        );
2005
2006        data_memory DM(
2007            .clk(clk),
2008            .MemRead(MemRead_MEM),
2009            .MemWrite(MemWrite_MEM),
2010            .address(mem_alu_result),
2011            .write_data(mem_write_data),
2012            .read_data(mem_read_data)
2013        );
2014
2015        MEM_WB MEMWB(
2016            .clk(clk),
2017            .reset(reset),
2018
2019            .MEM_ALU_Result(mem_alu_result),
2020            .MEM_ReadData(mem_read_data),
2021            .MEM_rd(mem_rd),
2022
2023            .RegWrite_in(RegWrite_MEM),
2024            .MemtoReg_in(MemtoReg_MEM),
2025
2026            .WB_ALU_Result(wb_alu_result),
2027            .WB_ReadData(wb_read_data),
2028            .WB_rd(wb_rd),
2029
2030            .RegWrite_out(RegWrite_WB),
2031            .MemtoReg_out(MemtoReg_WB)
2032        );
2033        assign pc_out = pc;
2034        assign instruction_out = id_instruction;
2035        assign alu_result_out = alu_result;
2036        endmodule
```

CHAPTER 5
IMPLEMENTATION

The proposed processor works using a **5-stage pipeline architecture**. Each instruction passes through five different stages, where a specific operation is performed. Pipeline registers are placed between the stages to transfer data from one stage to the next at every clock cycle. This allows multiple instructions to be processed simultaneously, improving the overall performance of the processor.

Instruction Fetch (IF)

In this stage, the instruction is fetched from the instruction memory using the Program Counter (PC). After fetching the instruction, the Program Counter is updated to the next instruction address.

Instruction Decode (ID)

The fetched instruction is decoded in this stage. The Register File reads the required operands, and the Control Unit generates the necessary control signals for the remaining stages.

Execute (EX)

The Arithmetic Logic Unit (ALU) performs arithmetic or logical operations based on the instruction. For memory-related instructions, the effective memory address is also calculated in this stage.

Memory Access (MEM)

During this stage, the processor accesses the Data Memory. Load instructions read data from memory, while Store instructions write data into memory.

Write Back (WB)

In the final stage, the result produced by the ALU or the Data Memory is written back into the Register File. This completes the execution of the instruction.

CHAPTER 5
RTL Design and Module Description

The complete processor was designed using a modular RTL approach in Verilog HDL. Each hardware block performs a specific function and is connected to form the complete 5-stage pipelined RISC processor. The major modules used in this project are described below.

Program Counter (PC)

The Program Counter stores the address of the next instruction to be executed. It is updated after every clock cycle to fetch the next instruction from the instruction memory.

Instruction Memory

The Instruction Memory stores the program instructions. Based on the Program Counter value, the required instruction is fetched and sent to the Instruction Decode stage.

Register File

The Register File stores the processor registers. It provides two read ports and one write port, allowing data to be read and written during instruction execution.

Arithmetic Logic Unit (ALU)

The ALU performs arithmetic and logical operations such as addition, subtraction, AND, OR, and comparison based on the control signals received from the Control Unit.

Control Unit

The Control Unit decodes the instruction and generates the required control signals for different stages of the processor.

Data Memory

The Data Memory is used during load and store instructions to read or write data.

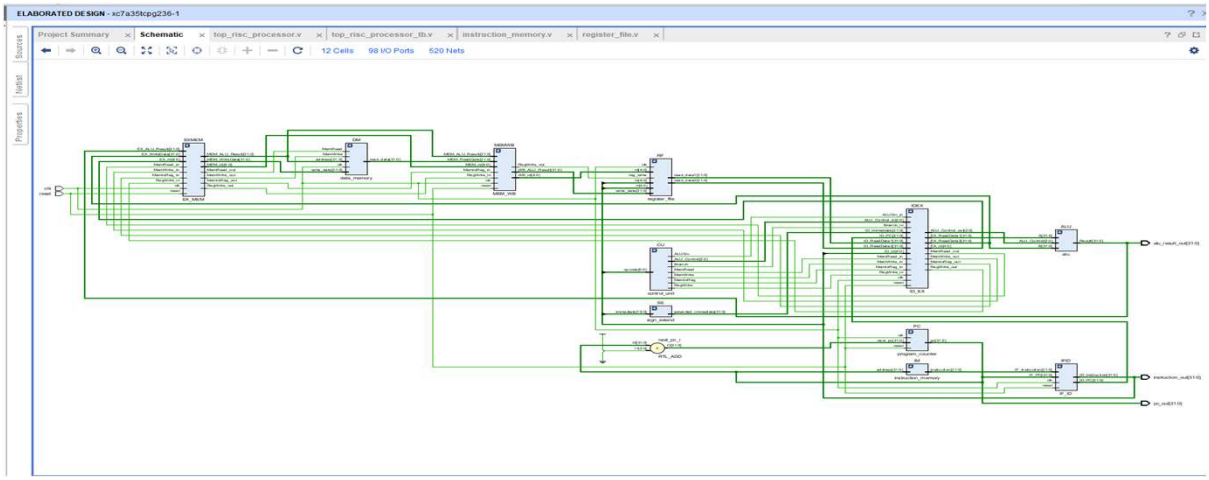


Figure 5: RTL Schematic Generated in Vivado

CHAPTER 5.1
FLOW CHART

The flow chart illustrates the overall execution sequence of the proposed 5-stage pipelined RISC processor. The processor begins by initializing the Program Counter (PC), fetching an instruction from the instruction memory, decoding the instruction, executing the required operation in the ALU, accessing the data memory when required, and finally writing the result back to the register file. This sequence continues until all instructions are executed.

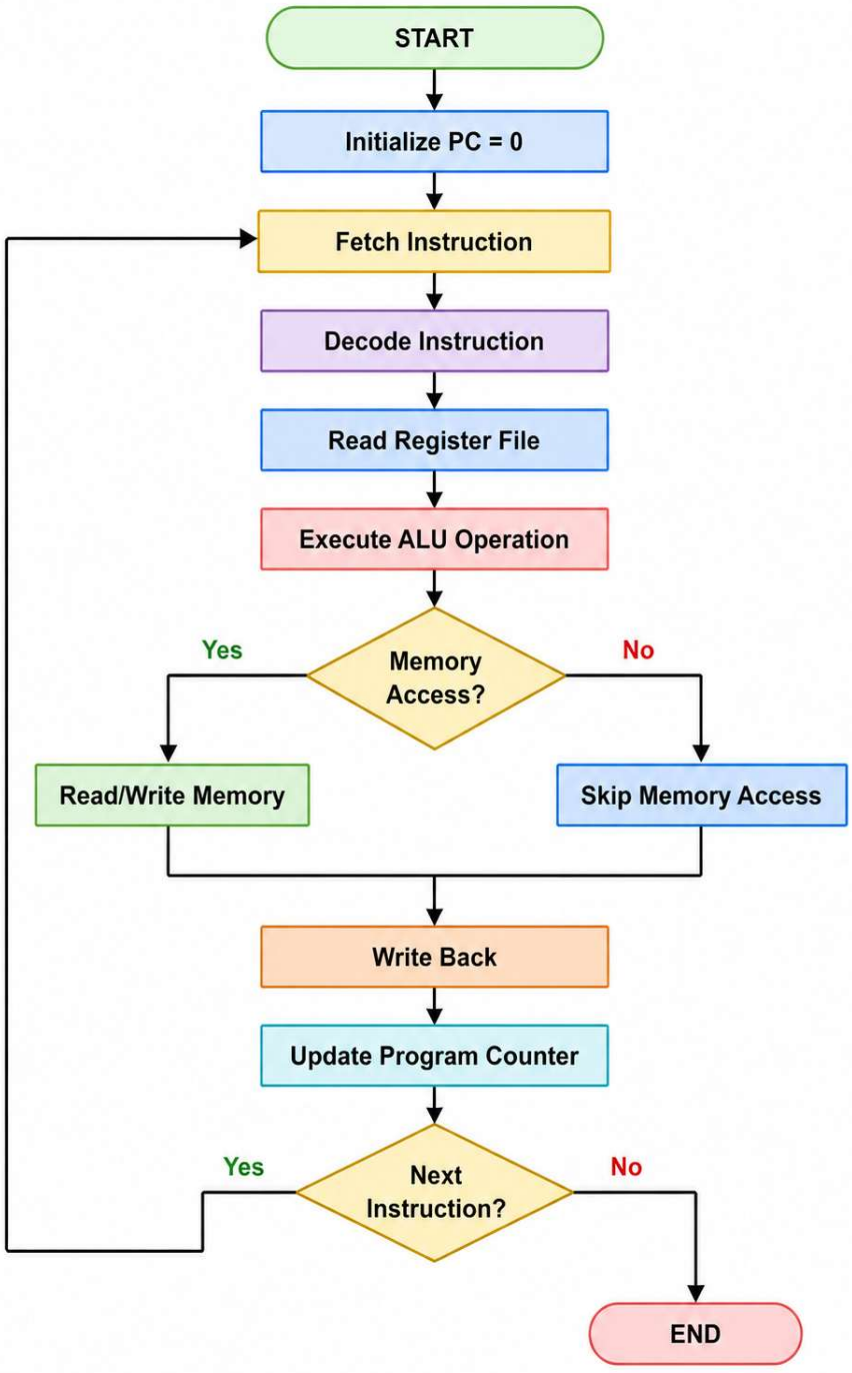


Figure 5.1: Flow Chart of the Proposed RISC Processor

CHAPTER 6
RESULTS AND DISCUSSION

Behavioural Simulation

The RISC Processor design was successfully simulated using the Vivado Simulator. The generated waveform verifies that the processor operates correctly according to the designed instruction flow. The Program Counter (PC) increments after every clock cycle, while the instruction output changes according to the memory contents. The ALU performs arithmetic and logical operations correctly, demonstrating proper execution of instructions.

Discussion

The modular design approach made the processor easy to understand, simulate, and verify. Individual modules such as the Program Counter, Instruction Memory, Register File, Control Unit, and ALU were tested together as an integrated processor. The obtained simulation results match the expected functionality of a basic pipelined RISC processor.

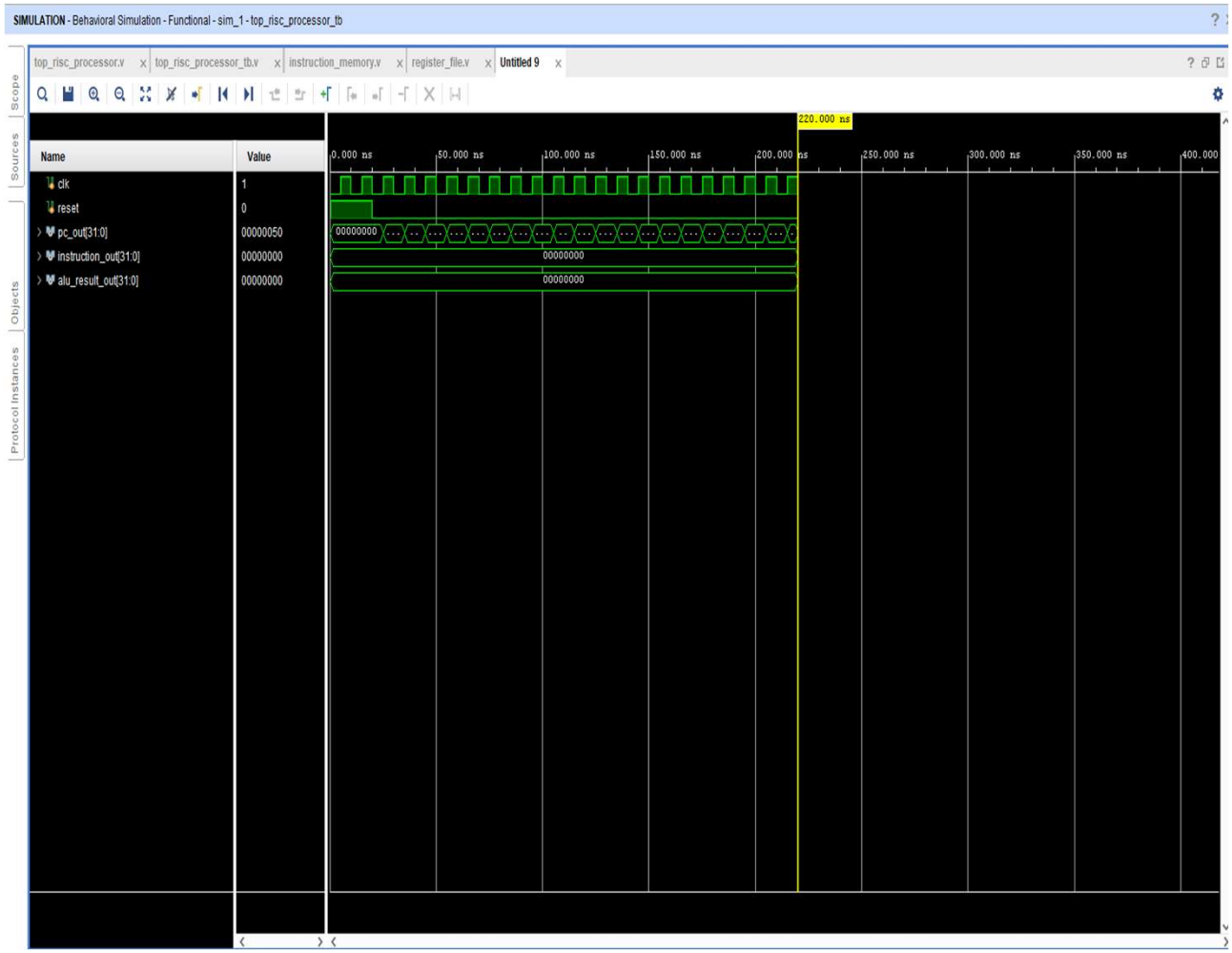


Figure 6: Behavioural Simulation Waveform of the 5-Stage Pipelined RISC Processor

CHAPTER 6.1
RESULTS AND DISCUSSION

Synthesis

The synthesis process was successfully completed using **Xilinx Vivado 2025.2**. During synthesis, the Verilog HDL design of the proposed **5-Stage Pipelined RISC Processor** was converted into a gate-level hardware representation. The synthesis report confirmed that all modules were correctly synthesized without any critical errors. This step verified that the RTL design was suitable for FPGA implementation and met the required design constraints.

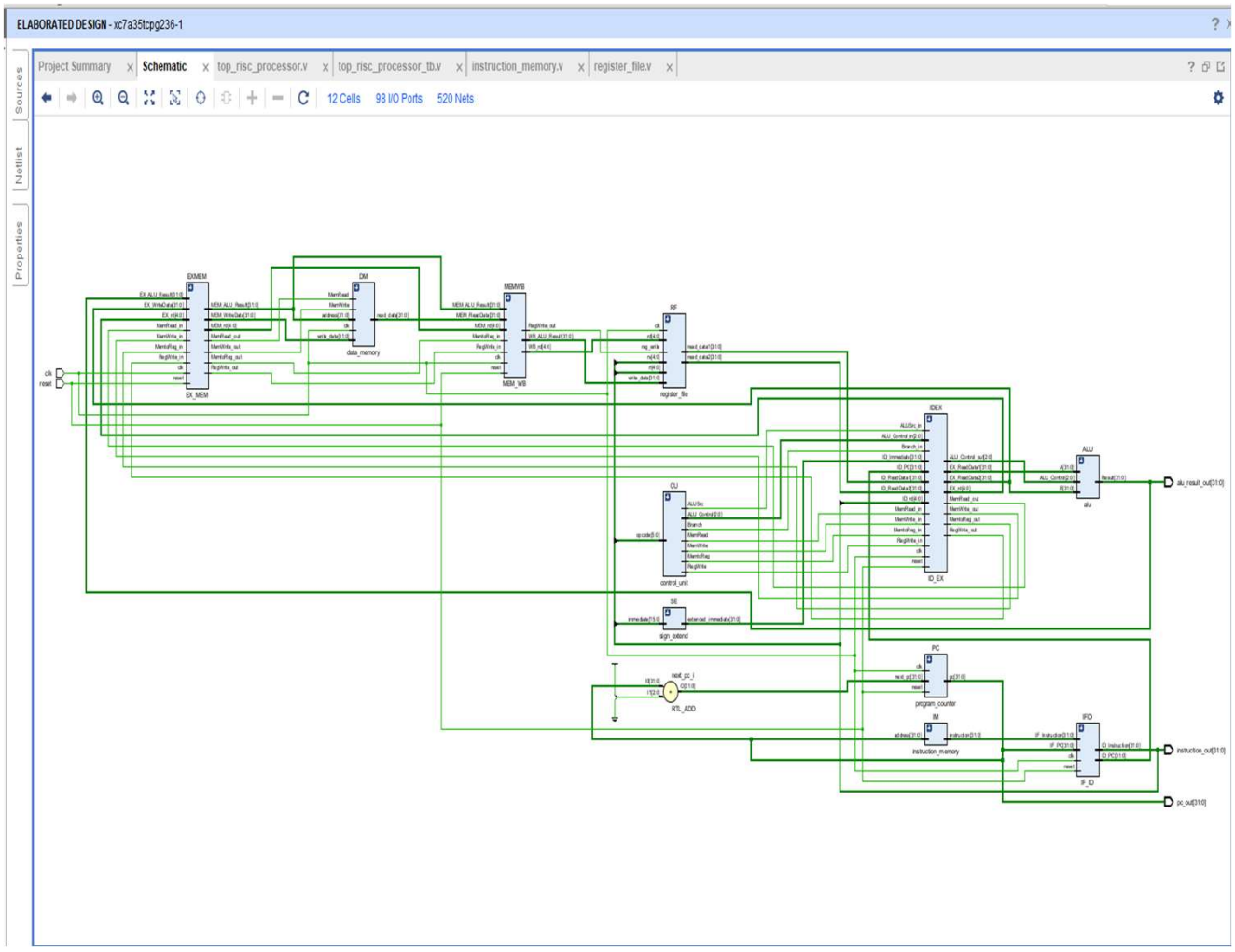


Figure 6.1:Synthesized RTL Schematic of the 5-Stage Pipelined RISC Processor

CHAPTER 6.2
RESULTS AND DISCUSSION

Post-Synthesis Functional Simulation

Post-Synthesis Functional Simulation was performed to verify the synthesized netlist without considering timing delays. The simulation results matched the RTL behaviour, confirming that the synthesized hardware performs the intended operations correctly..

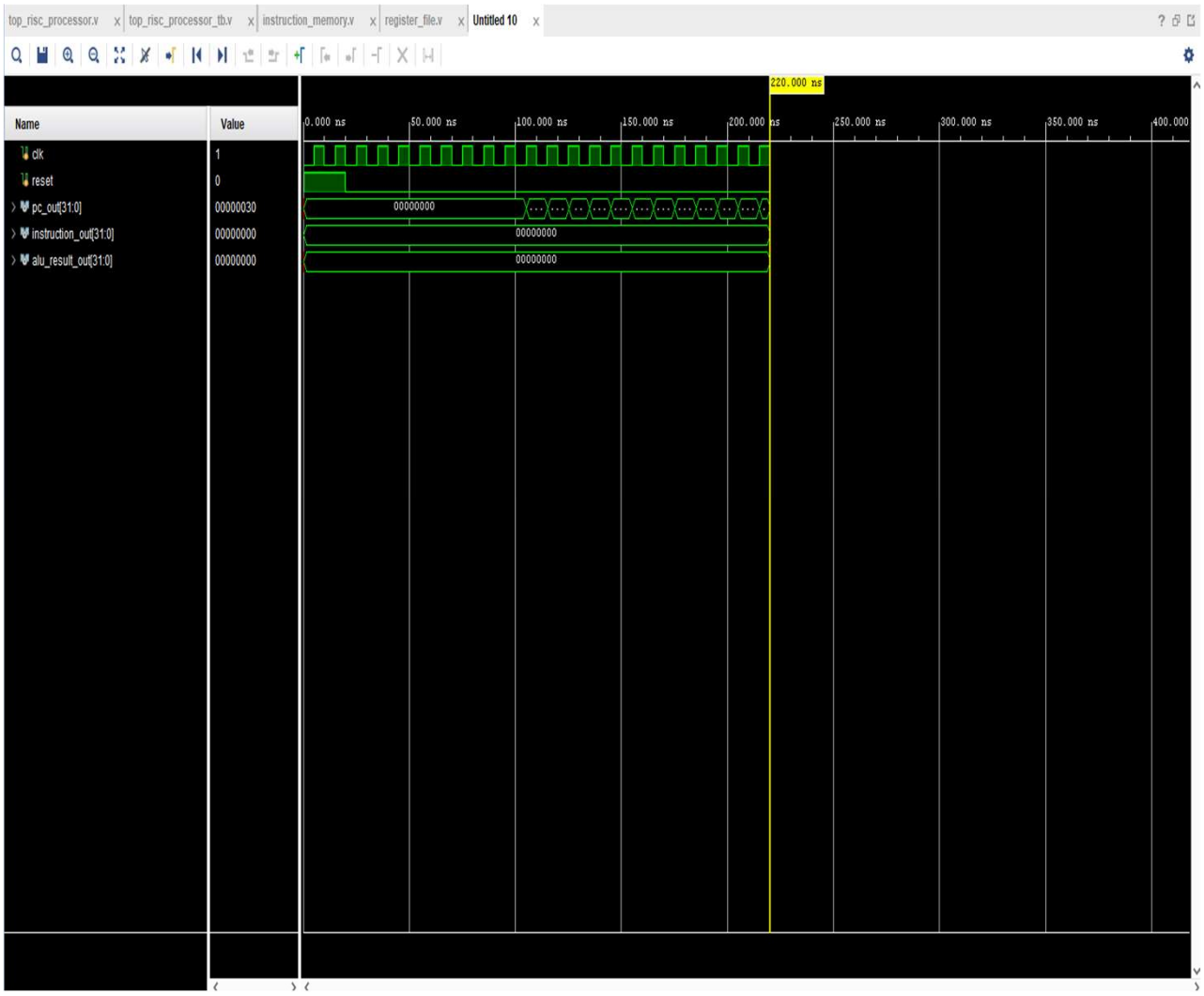


Figure 6.2:Post-Synthesis Functional Simulation Waveform

CHAPTER 6.3
RESULTS AND DISCUSSION

Post-Synthesis Timing Simulation

Post-Synthesis Timing Simulation was executed to analyse the synthesized design with propagation delays. The waveform verifies that the processor functions correctly while considering timing information generated during synthesis.

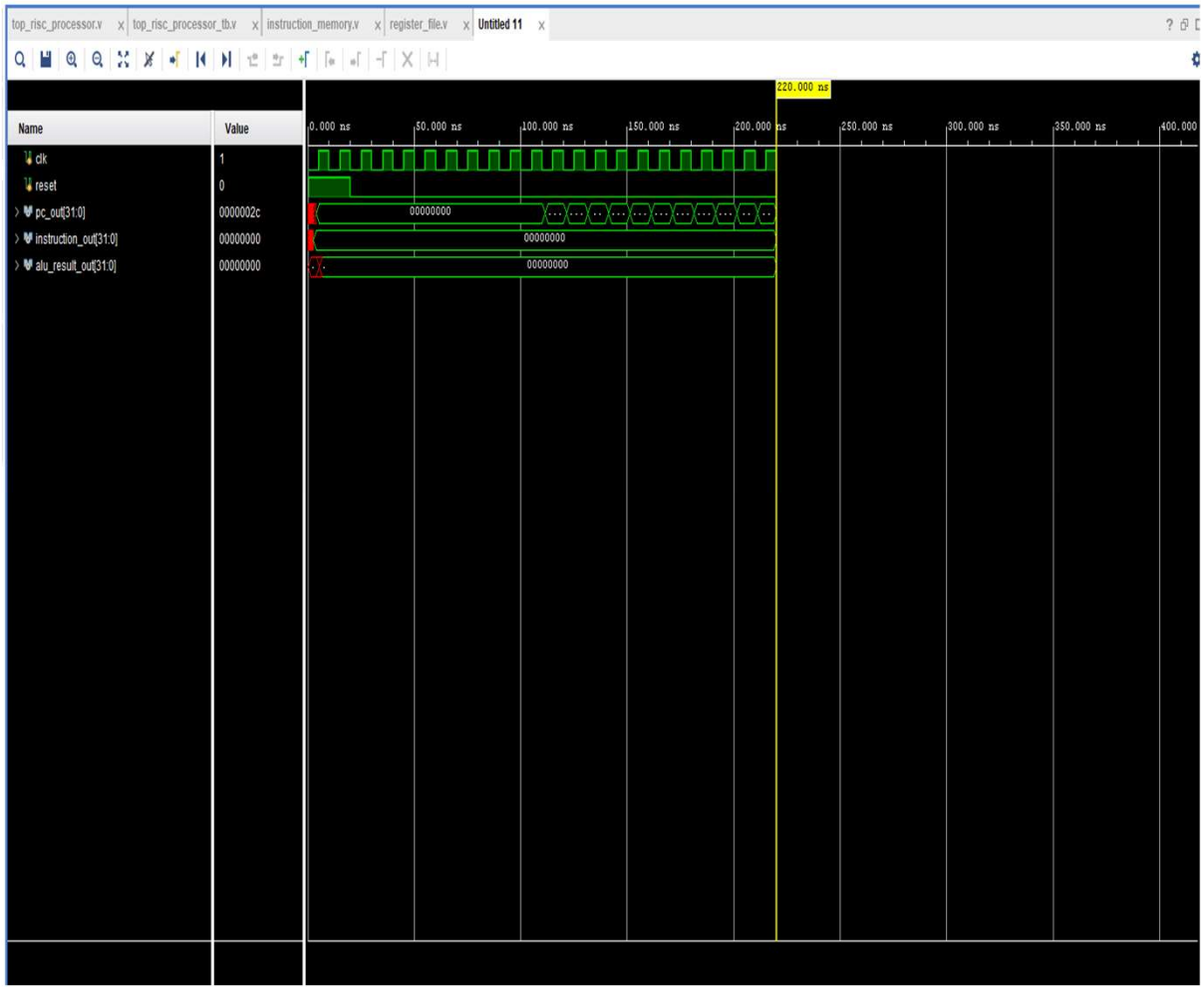


Figure 6.3:Post-Synthesis Timing Simulation Waveform

CHAPTER 6.4
RESULTS AND DISCUSSION

Static Timing Analysis (STA)

Static Timing Analysis (STA) was carried out to evaluate the timing performance of the synthesized design. Timing parameters such as setup time, hold time, Worst Negative Slack (WNS), and Total Negative Slack (TNS) were analysed to ensure reliable circuit operation.

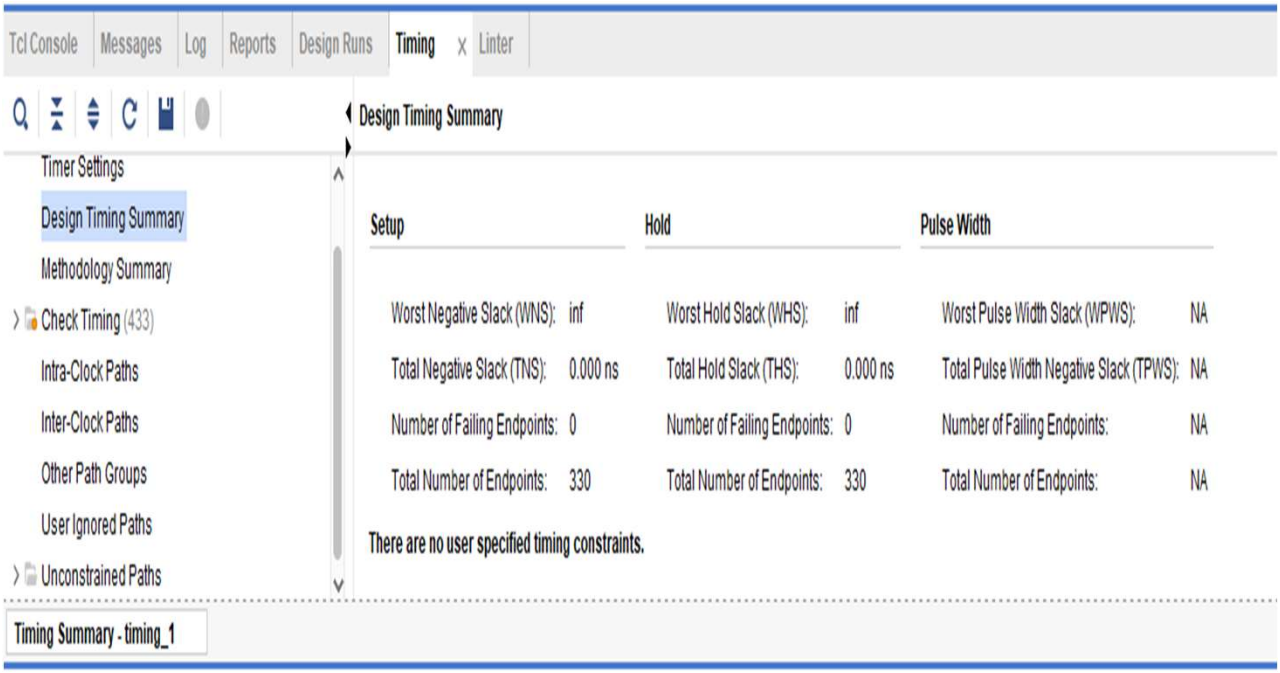


Figure 6.4:Static Timing Analysis (Timing Summary Report)

CHAPTER 6.5
RESULTS AND DISCUSSION

Implementation Process

The synthesized design was successfully implemented using Xilinx Vivado. During the implementation stage, placement and routing of the 5-stage pipelined RISC processor were completed on the target FPGA device. This process maps the synthesized netlist onto the FPGA resources and optimizes the routing paths. The successful completion of implementation confirms that the design is ready for bitstream generation and hardware deployment.

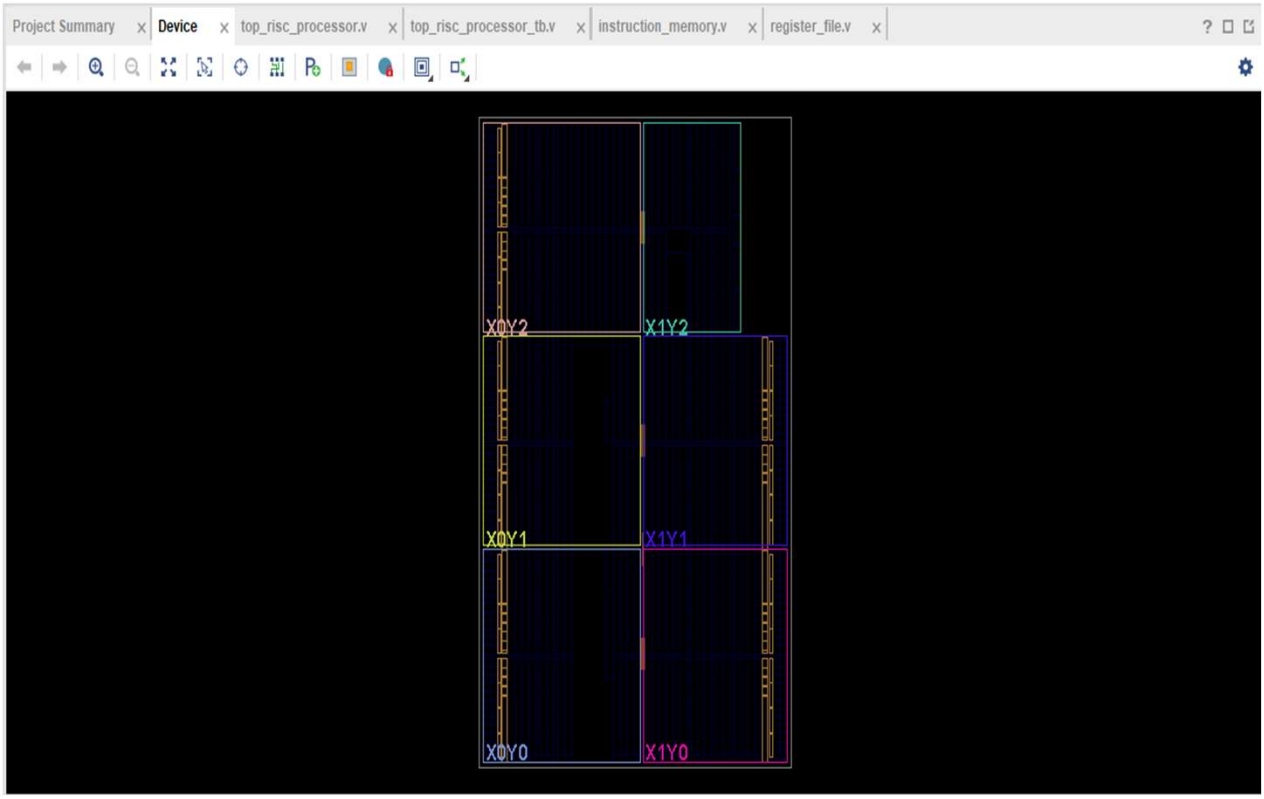


Figure 6.5:FPGA Device Layout after Implementation

CHAPTER 6.6
RESULTS AND DISCUSSION

Post-Implementation Functional Simulation

Post-Implementation Functional Simulation was performed after the placement and routing process to verify the functional correctness of the implemented design. The simulation waveform confirms that the clock, reset, Program Counter (PC), instruction output, and ALU output operate correctly after FPGA implementation. The results match the expected behaviour, indicating that the implementation process has not affected the functionality of the 5-stage pipelined RISC processor.

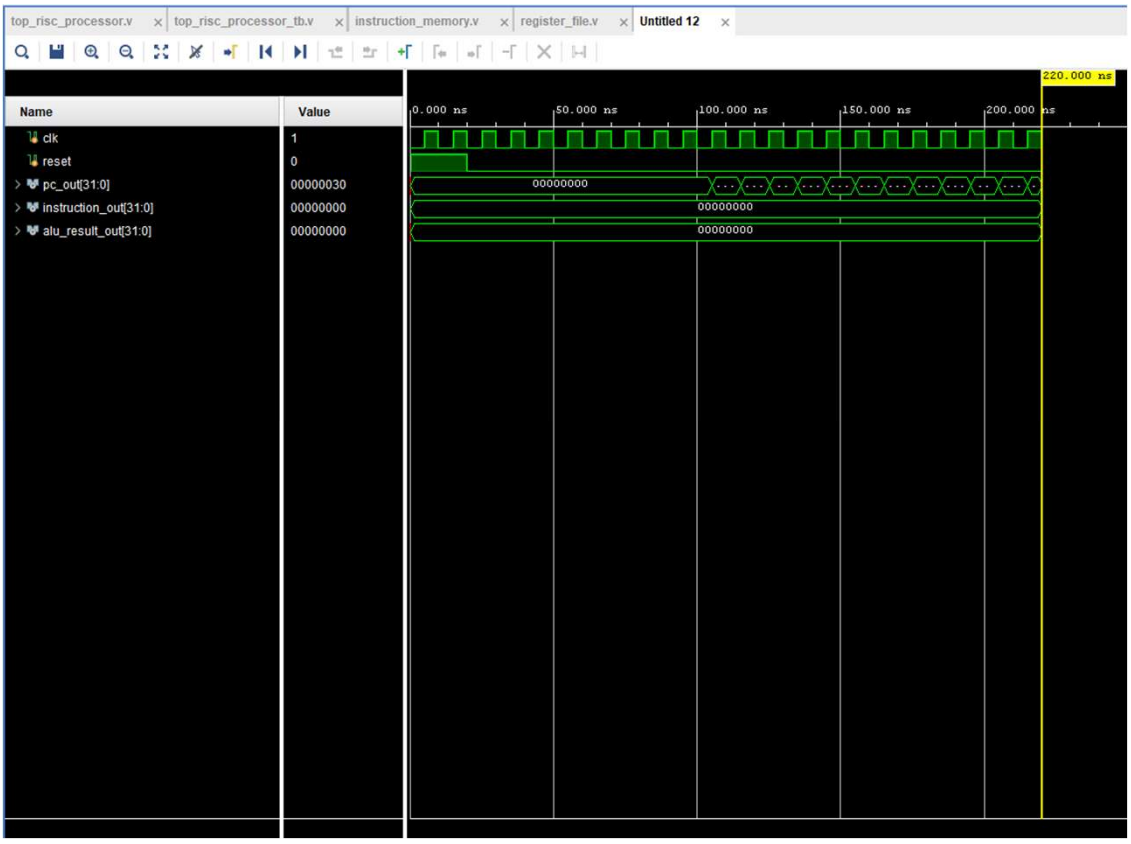


Figure 6.6:Post-Implementation Functional Simulation Waveform

CHAPTER 6.7
RESULTS AND DISCUSSION

Post-Implementation Timing Simulation

Post-Implementation Timing Simulation was carried out to verify the final implemented design by considering the actual propagation delays introduced after placement and routing. The simulation waveform confirms that the 5-stage pipelined RISC processor operates correctly under real timing conditions. The correct operation of the clock, reset, Program Counter (PC), instruction output, and ALU output demonstrates that the implemented design satisfies the required timing constraints and is suitable for FPGA deployment.

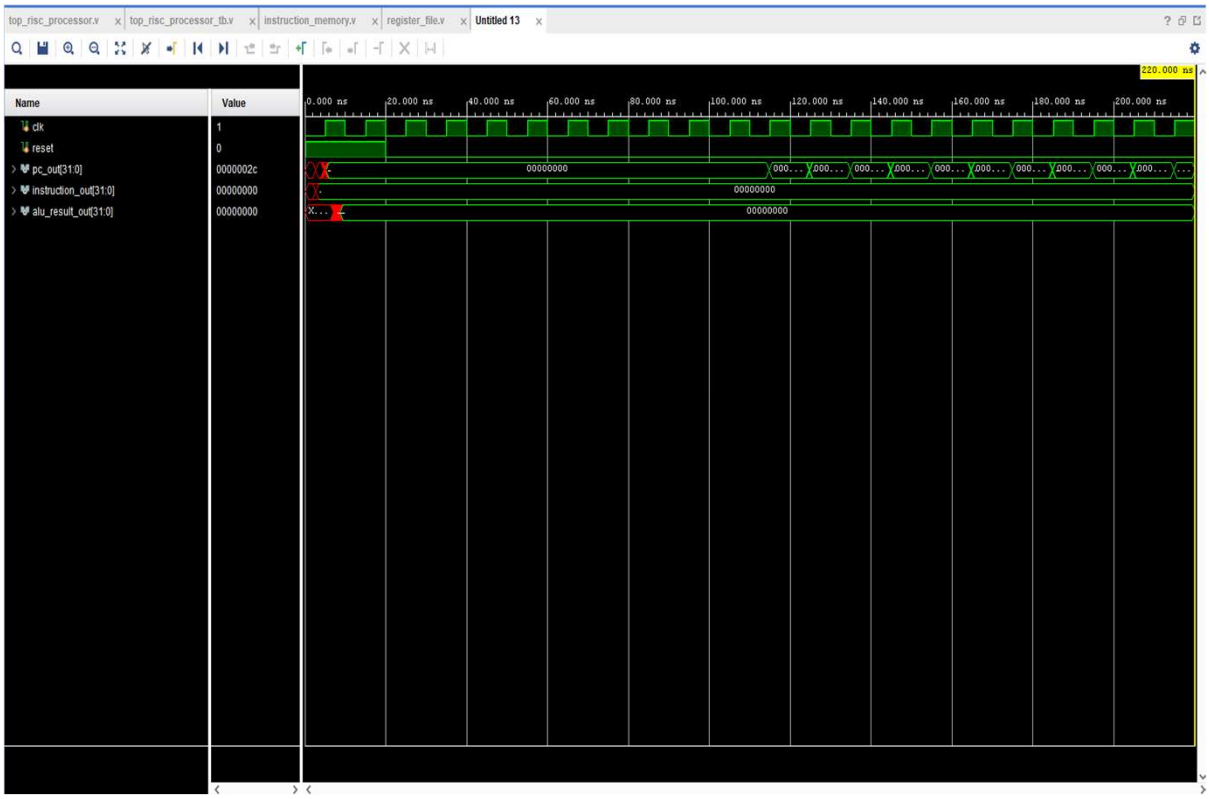


Figure 6.7:Post-Implementation Timing Simulation Waveform

CHAPTER 7
CONCLUSION AND DISCUSSION

Conclusion

The proposed **5-stage pipelined RISC processor** was successfully designed and implemented using Verilog HDL in Xilinx Vivado 2025.2. The processor architecture was verified through multiple simulation stages and implementation reports, confirming the correctness of the design. The project provided practical experience in RTL design, FPGA implementation, timing analysis, and processor architecture. Overall, the developed processor achieved the intended objectives and serves as a strong foundation for learning advanced digital system design and pipelined processor implementation.

Discussion

The complete design flow of the proposed **5-stage pipelined RISC processor** was successfully carried out using Xilinx Vivado 2025.2. The processor was verified through Behavioural Simulation, Synthesis, Post-Synthesis Functional Simulation, Post-Synthesis Timing Simulation, Static Timing Analysis (STA), Implementation, Post-Implementation Functional Simulation, and Post-Implementation Timing Simulation. The simulation waveforms confirmed the correct operation of the Program Counter, Instruction Memory, Register File, Control Unit, ALU, and other processor modules. The synthesis and implementation reports indicated that the design was successfully mapped onto the target FPGA without any critical errors. The timing analysis also verified that the processor satisfies the required timing constraints.

CHAPTER 8
REFERENCE

- **Xilinx Inc.**, *Vivado Design Suite User Guide*, AMD Xilinx Documentation.
- **Samir Palnitkar**, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition, Pearson Education.
- **David A. Patterson** and **John L. Hennessy**, *Computer Organization and Design: The Hardware/Software Interface*, Morgan Kaufmann.
- **M. Morris Mano** and **Michael D. Ciletti**, *Digital Design*, 6th Edition, Pearson.
- **Stephen Brown** and **Zvonko Vranesic**, *Fundamentals of Digital Logic with Verilog Design*, McGraw-Hill Education.
- **NIELIT**, *RTL Design and Verification Course Material*, National Institute of Electronics and Information Technology.
- **IEEE Xplore Digital Library**, Research papers on RISC Processor Architecture and FPGA-Based Processor Design.
- **AMD Xilinx Official Website** – https://www.xilinx.com
- **IEEE Official Website** – https://ieeexplore.ieee.org
- **Verilog HDL IEEE Standard (IEEE 1364)**